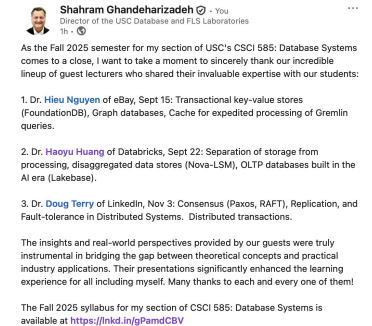


Announcements

- Exam 2 is 3 weeks from today. Do not fall behind!
 - I have posted all assigned papers on the USC Brightspace. They should be visible to you.
- Doug Terry gave a very nice lecture a week ago. I will cover RAFT in greater detail today.
- I posted a note on LinkedIn, thanking all guest lecturers for this semester. I want them to feel appreciated - high number of likes, please! If you are on LinkedIn and learned from the guest lecturers then pls visit and give a like.





USC Viterbi
School of Engineering

Strong Consistency in Cache Augmented SQL Systems.

Shahram Ghandeharizadeh, Jason Yap, Hieu Nguyen



Outline



- Motivation
- Strong Consistency
- IQ Framework
- Future Research

Simple Operations

- Operations that read and write a small amount of data
 - Challenge: High volume of requests with a low latency requirement.
- Person-to-person service providers in 1 minute:



2.4M requests



184K page views



3.3M snaps



97M Messages



Facebook, <http://thenextweb.com/facebook/2014/10/28/facebook-1-35-billion-users/>
Google, <http://expandedramblings.com/index.php/google-plus-statistics/>
Twitter, <https://about.twitter.com/company>
Wikipedia, <http://stats.wikimedia.org/EN/Sitemap.htm>

Numbers from 2024-2025.

Slow RDBMSs

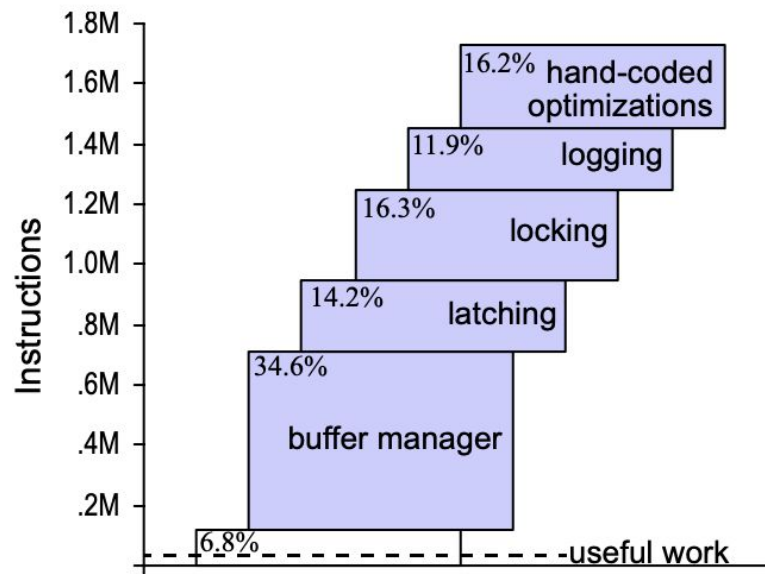


Figure 1. Breakdown of instruction count for various DBMS components for the New Order transaction from TPC-C. The top of the bar-graph is the original Shore performance with a main memory resident database and no thread contention. The bottom dashed line is the useful work, measured by executing the transaction on a no-overhead kernel.



Slow RDBMSs + Fast Look Aside Cache

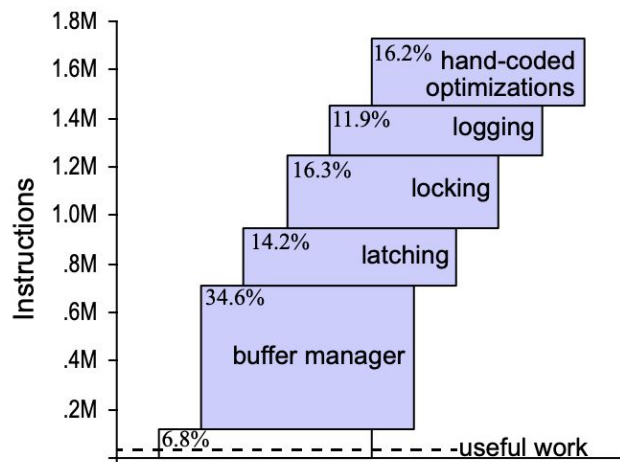


Figure 1. Breakdown of instruction count for various DBMS components for the New Order transaction from TPC-C. The top of the bar-graph is the original Shore performance with a main memory resident database and no thread contention. The bottom dashed line is the useful work, measured by executing the transaction on a no-overhead kernel.



Look aside cache with LRU/CAMP eviction:

- Memory only - no buffer pool manager,
- Network protocol for key-value transmission,
- No logging,
- No locking.

Slow RDBMSs + Fast Look Aside Cache

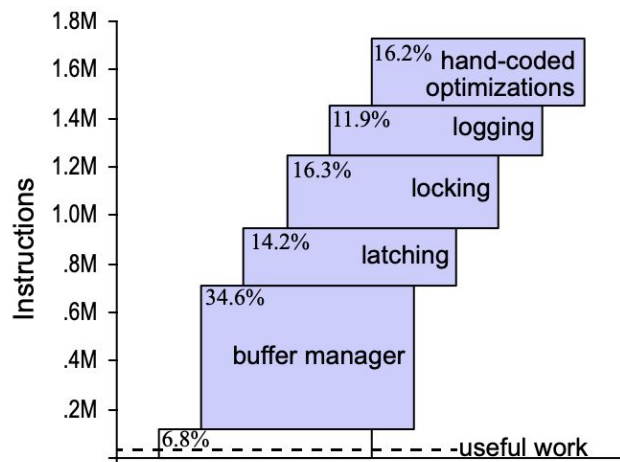


Figure 1. Breakdown of instruction count for various DBMS components for the New Order transaction from TPC-C. The top of the bar-graph is the original Shore performance with a main memory resident database and no thread contention. The bottom dashed line is the useful work, measured by executing the transaction on a no-overhead kernel.



MySQL
Oracle

...



memcached
Redis

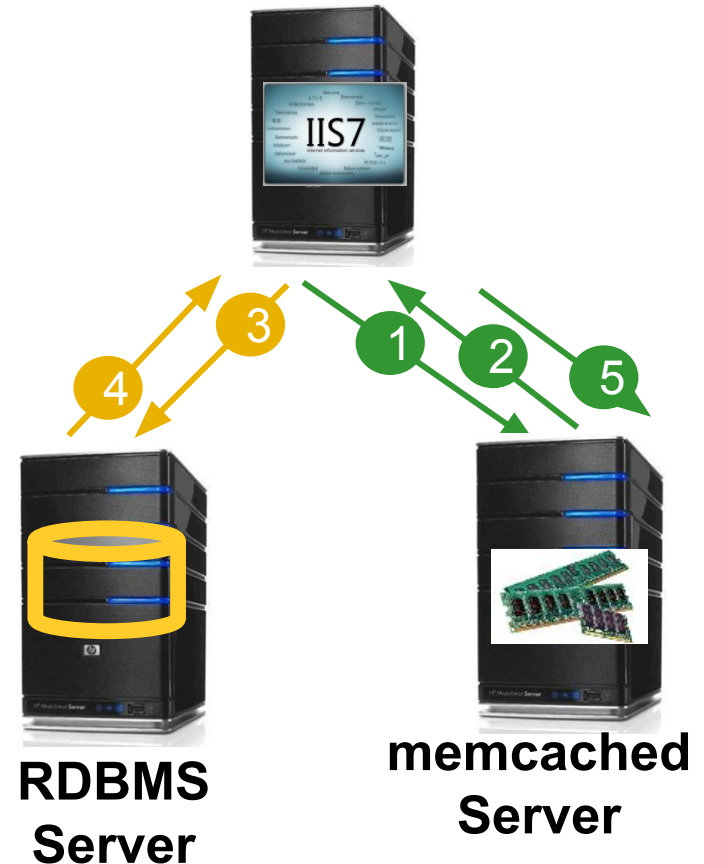
...

Look aside cache with
LRU/CAMP eviction:

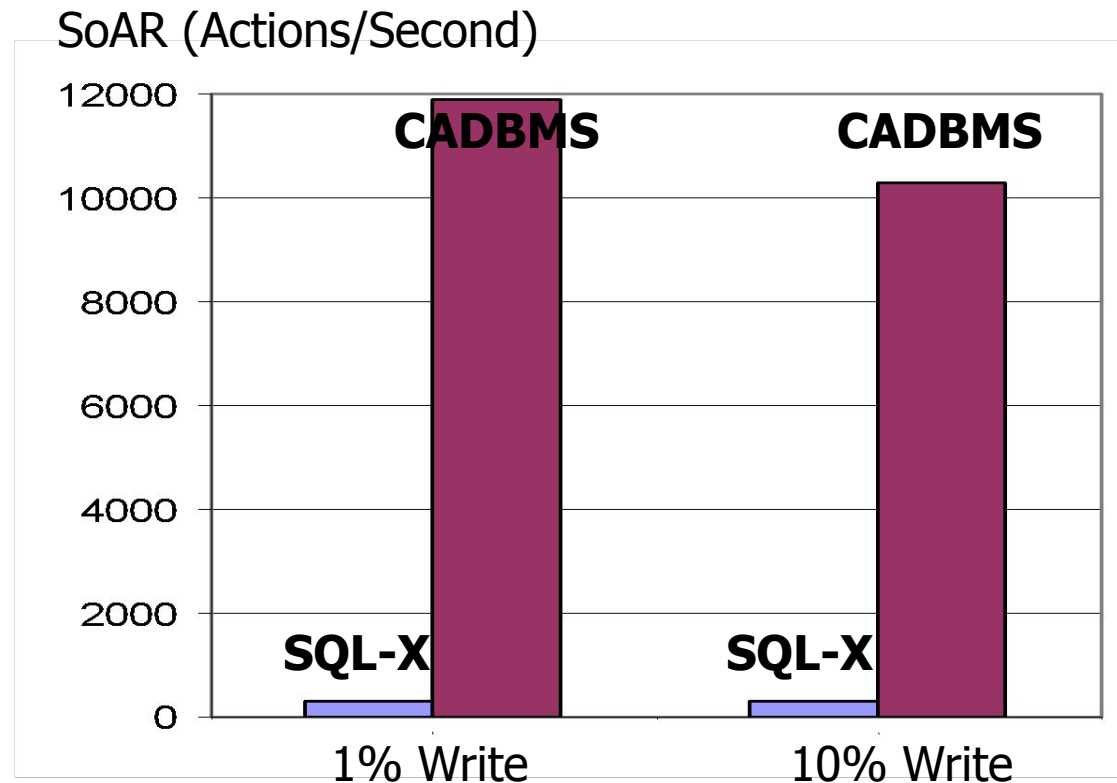
- Memory only - no buffer pool manager,
- Network protocol for key-value transmission,
- No logging,
- No locking.

Cache Augmented DBMSs

1. Value = Get (Key)
2. If Value is found, go to Step 6.
3. SQL queries
4. Process query results
 - ☐ Application constructs Value using the results
5. Put(Key, Value)
6. Use Value to generate HTML result page



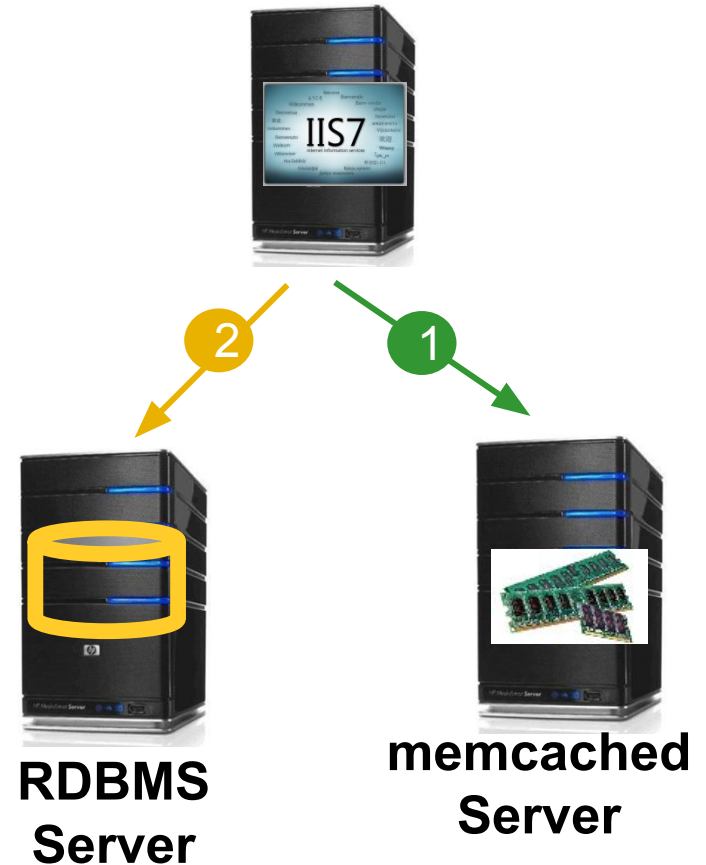
Why?



- SLA is 95% of actions to observe a response time faster than 100 msec.
- Requirement: High cache hit rate, e.g., 99%.

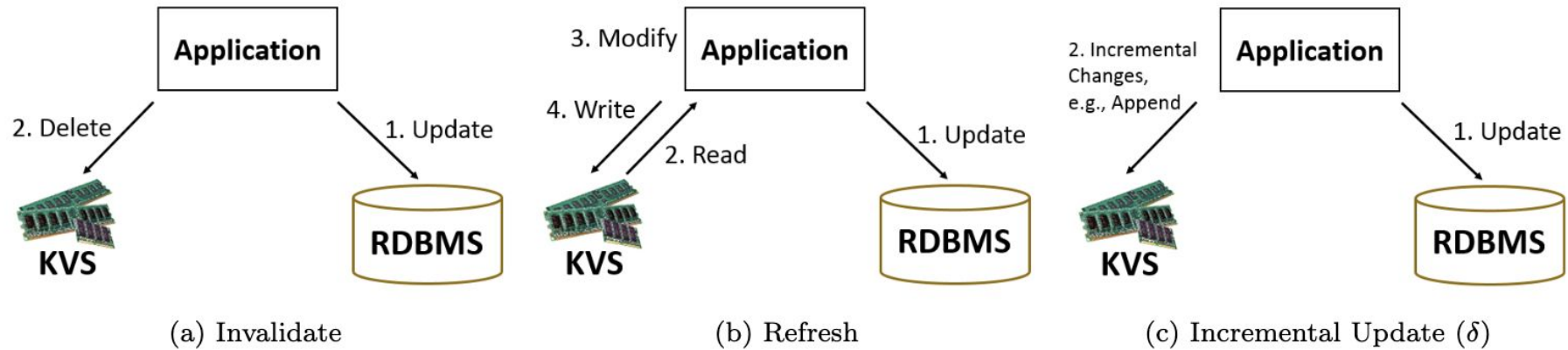
CADBMS: Update

- SQL DML Command:
Insert, Delete, Update
- Several paradigms:
Invalidate, Refresh (Refill),
and incremental update.



- SQL Data Manipulation Language, **DML**: Insert, delete, update
- A **session** is a sequence of RDBMS operations as 1 transaction followed by one or more KVS operations.
 - Order is not important.
 - A session acquires leases in a manner similar to 2-phase locking.
 - More on sessions later in the presentation.
- A Key Value Store, **KVS**, refers to the cache.
- Three approaches to update the KVS (cache): Invalidate, Refresh, or Incrementally update.

Invalidate, Refresh, Incremental Update



System Load	Invalidate	Refresh	Incremental Update
1 session	0%	0%	0%
Low 10 concurrent sessions	0.5%	0%	0.01%
Moderate 100 concurrent sessions	1.1%	1.4%	0.2%
High 200 concurrent sessions	1.3%	1.8%	2.9%

Table 1: Percentage of unpredictable data with Twemcache. These percentages are reduced to zero with the IQ framework.

Example: Social Networking

RDBMS Tables:

User(id, name, email, address, URL, ...)

PendingFriends (InviterID, InviteeID, 1)

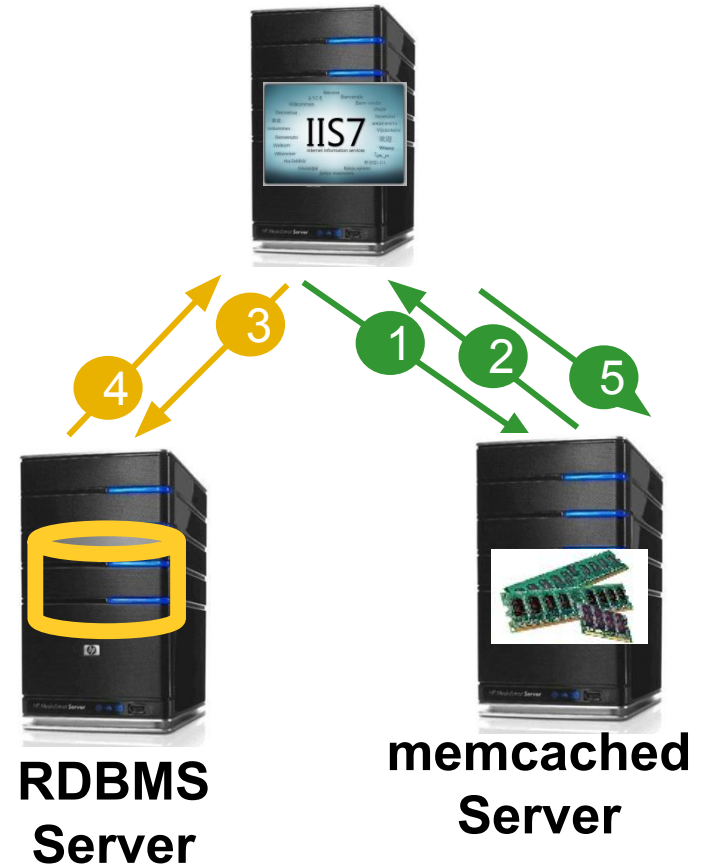
Profile consists of: user information, number of pending friends

Query 1: Select * from Users where id = uid

Query 2: Select count(*) from PendingFriends where InviteeID=uid

Key = "Profile"+uid

Value = {id=101, name="Shahram", ..., PendingFriendCount=10, ...}



Invalidate

Invite Friend(InviterID, InviteeID)

1. Key = "Profile"+InviteeID

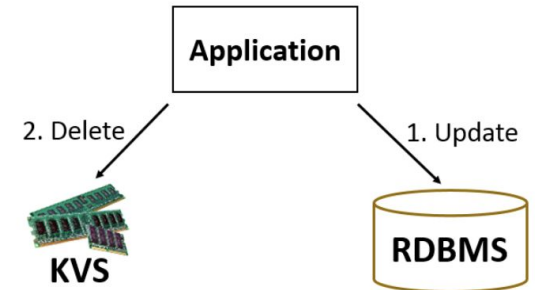
2. KVS.Delete(key)

3. Begin RDBMS Xact

a. Insert(InviterID, InviteeID, 1) in PendingFriends

b. Update PendingFriendCount of InviteeID by 1 in the Users table

4. Commit Xact



Invalidate

Invite Friend(InviterID, InviteeID)

1. Key = "Profile"+InviteeID

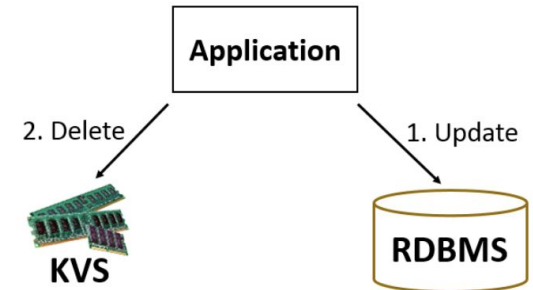
2. KVS.Delete(key)

3. Begin RDBMS Xact

a. Insert(InviterID, InviteeID, 1) in PendingFriends

b. Update PendingFriendCount of InviteeID by 1 in the Users table

4. Commit Xact



Question: Does it matter if Step 2 is placed after Step 3?

Refresh/Refill

Invite Friend(InviterID, InviteeID)

1. Key = "Profile"+InviteeID

2. Vold = KVS.Read(Key)

3. Vnew = Increment Vold.#PendingFriends by 1

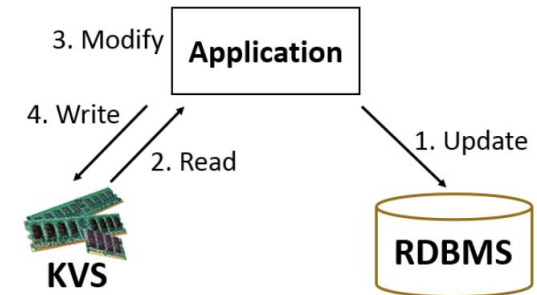
4. KVS.Write (Key, Vnew)

5. Begin RDBMS Xact

a. Insert(InviterID, InviteeID, 1) in PendingFriends

b. Update PendingFriendCount of InviteeID by 1 in the Users table

6. Commit Xact



Refresh/Refill: RMW

Invite Friend(InviterID, InviteeID)

1. Key = "Profile"+InviteeID

2. Vold = KVS.Read(Key)

3. Vnew = Increment Vold.#PendingFriends by 1

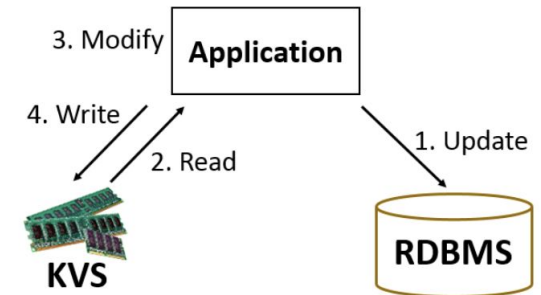
4. KVS.Write (Key, Vnew)

5. Begin RDBMS Xact

a. Insert(InviterID, InviteeID, 1) in PendingFriends

b. Update PendingFriendCount of InviteeID by 1 in the Users table

6. Commit Xact



Read-Modify-Write
RMW

Incremental Update

Invite Friend(InviterID, InviteeID)

1. Key = "Profile"+InviteeID

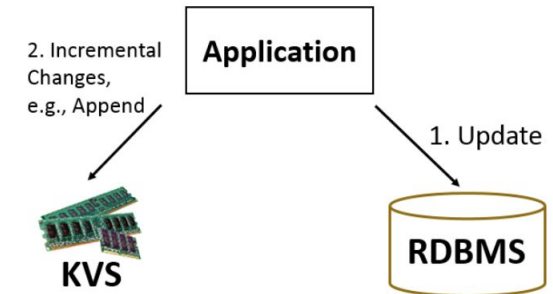
2. KVS.Increment(Key, 1)

3. Begin RDBMS Xact

a. Insert(InviterID, InviteeID, 1) in PendingFriends

b. Update PendingFriendCount of InviteeID by 1 in the Users table

4. Commit Xact



Incremental Update

Invite Friend(InviterID, InviteeID)

1. Key = "Profile"+InviteeID

2. KVS.Increment(Key, 1)

Increment is
implemented by the
KVS

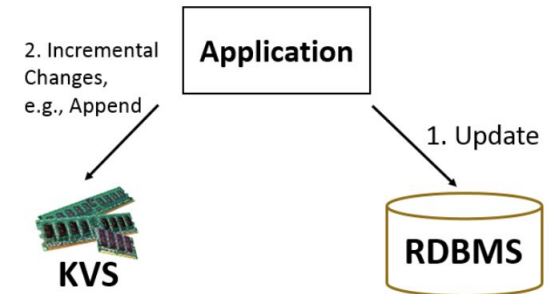
3. Begin RDBMS Xact

a. Insert(InviterID, InviteeID, 1) in PendingFriends

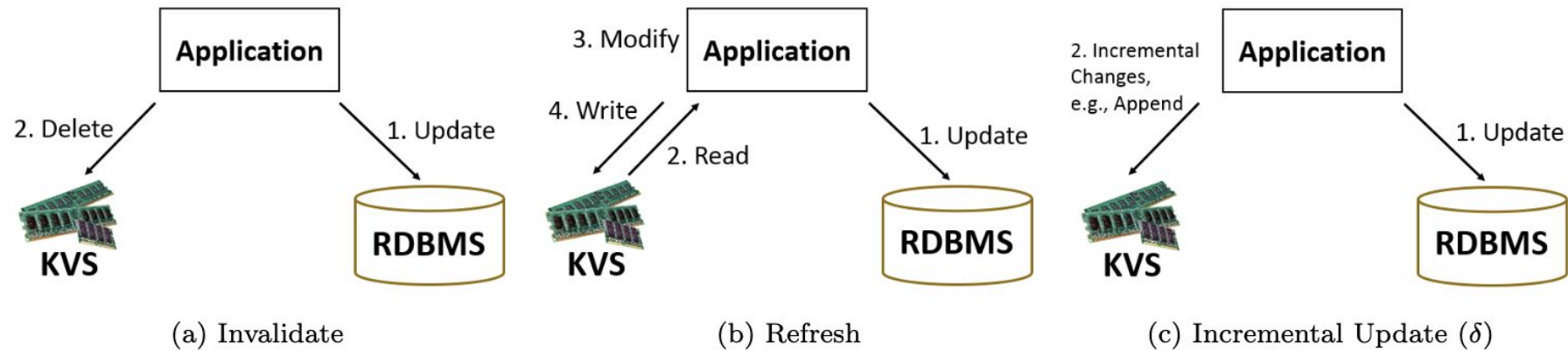
b. Update PendingFriendCount of InviteeID by 1 in the Users table

4. Commit Xact

KVS Increment
resembles SQL Update
command



Invalidate, Refresh, Incremental Update



Operation	Invalidate	Refresh	Incremental Update
Read	✓	✓	✓
Write	✓	✓	✓
Delete	✓	✓	✓
R-M-W	✗	✓	✗
δ	✗	✗	✓

Questions?

Question: Invalidate

Invite Friend(InviterID, InviteeID)

1. Key = "Profile"+InviteeID

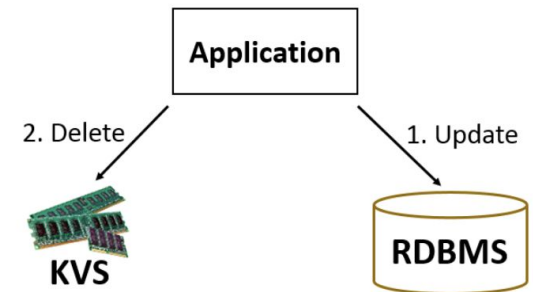
2. KVS.Delete(key)

3. Begin RDBMS Xact

a. Insert(InviterID, InviteeID, 1) in PendingFriends

b. Update PendingFriendCount of InviteeID by 1 in the Users table

4. Commit Xact



Why is it acceptable to delete the cache entry?

Invalidate

Invite Friend(InviterID, InviteeID)

1. Key = "Profile"+InviteeID

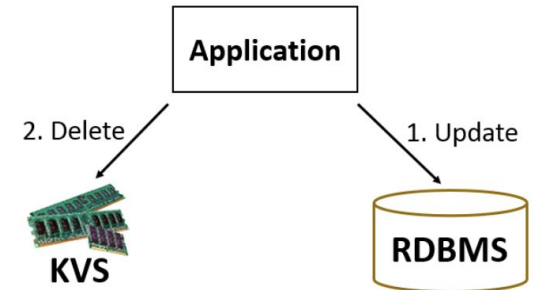
2. KVS.Delete(key)

3. Begin RDBMS Xact

a. Insert(InviterID, InviteeID, 1) in PendingFriends

b. Update PendingFriendCount of InviteeID by 1 in the Users table

4. Commit Xact



- When a cache entry is deleted, its next reference observes a miss, computes the missing value using the RDBMS and inserts it in the cache.
- Note: Multiple sessions deleting the same key is NOT a conflict.

Outline



- Motivation
- Strong Consistency
- IQ Framework
- Future Research

- ACID properties
- Consequences of violating strong consistency, e.g.,
 - ☐ Number of friends does not match the list shown
 - ☐ Reversed ordering of a comment and its response
 - ☐ A shopping cart with extra items
- Key insight: The RDBMS and the cache are not aware of one another & the application inserts stale data in the cache due to:
 - ☐ Undesirable race conditions [3]
 - ☐ Snapshot isolation [4]

[3] Ghandeharizadeh and Yap. Gumball. DBSocial 2012.

[4] Perez-Sorrosal et. al. Elastic SI-Cache. VLDB Journal 2011.

Undesirable Race Conditions

- Example:



Administrator

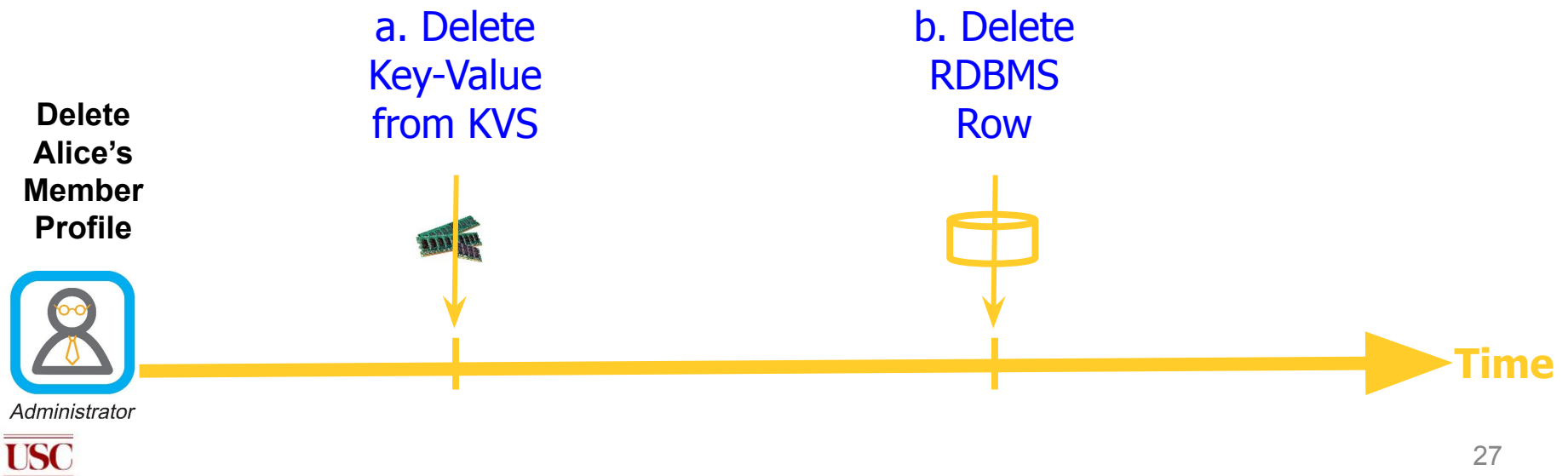
**Delete Alice's
Member Profile**



Alice Login

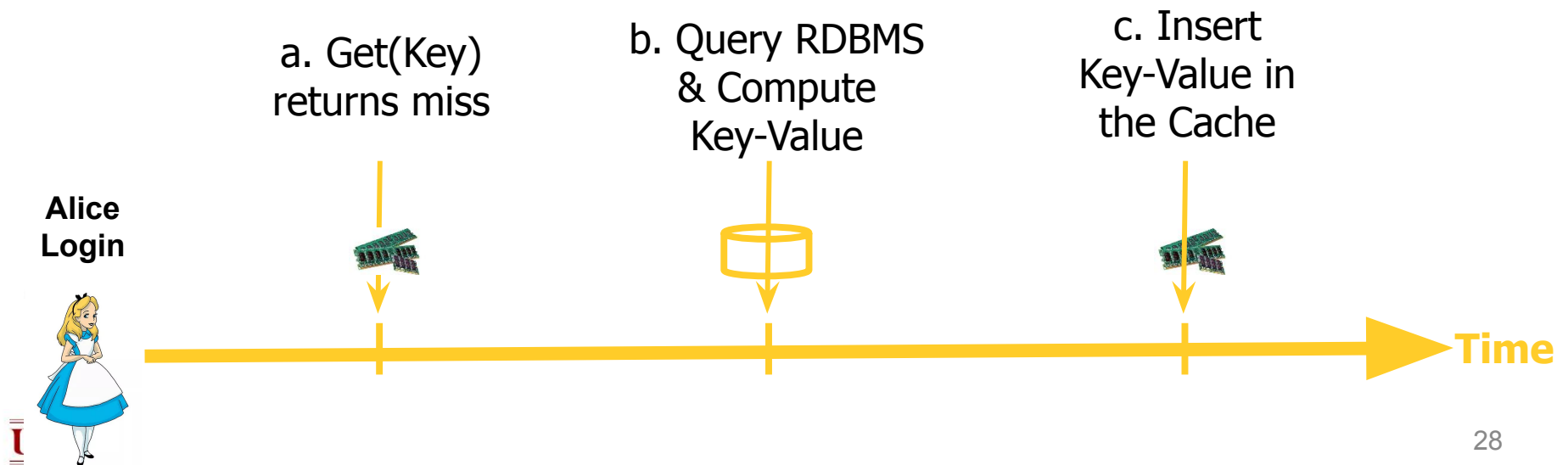
Example: Xact 1

- Site administrator deletes Alice's Profile page:
 - a. **Delete Alice's key-value pair(s) from the cache server.**
 - b. **Delete Alice's record(s) from the RDBMS.**

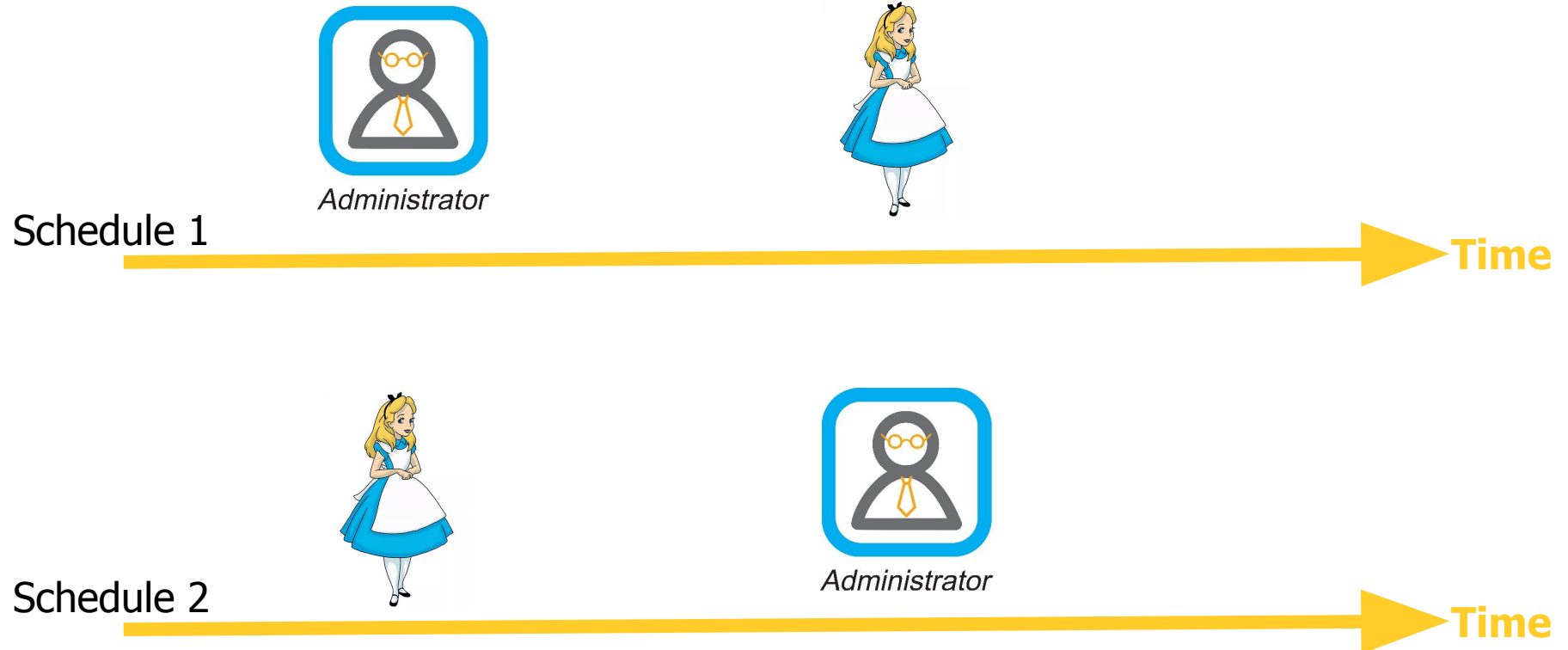


Example: Xact 2

- Alice looks up her profile and her request observes a cache miss.
 - a. Get(Key) returns miss
 - b. Issue queries to the RDBMS to compute Key-Value
 - c. Insert Key-Value in the cache



Objective: Two Possible Serial Schedules



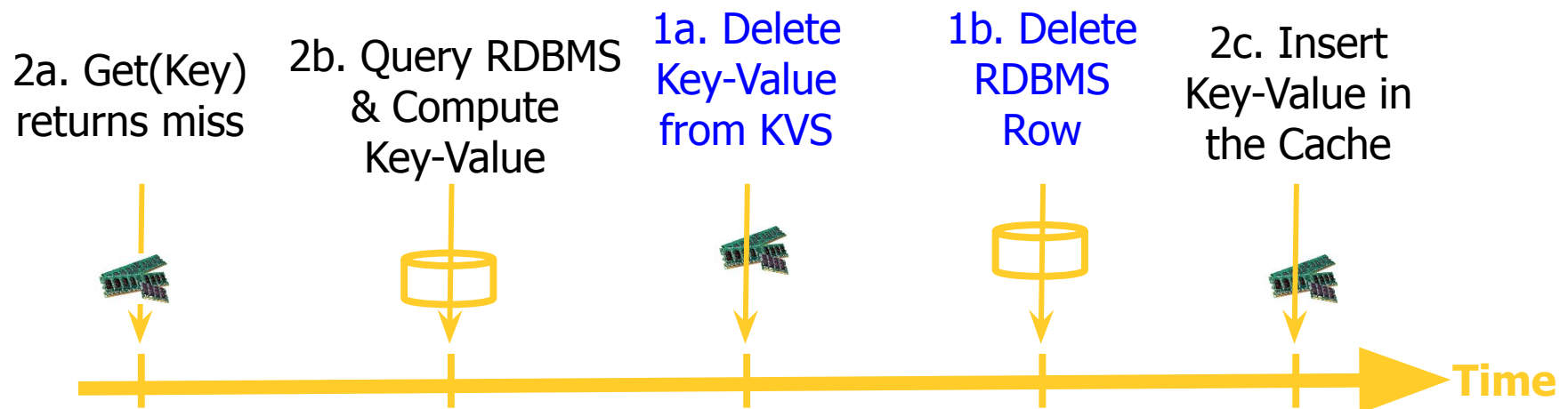
Serial Execution of Xacts 1 & 2



- Observations:
 - Alice is gone for good
 - A subsequent look up of Alice returns “Alice not found”
 - Cache and database are consistent with one another

Undesirable Race Condition

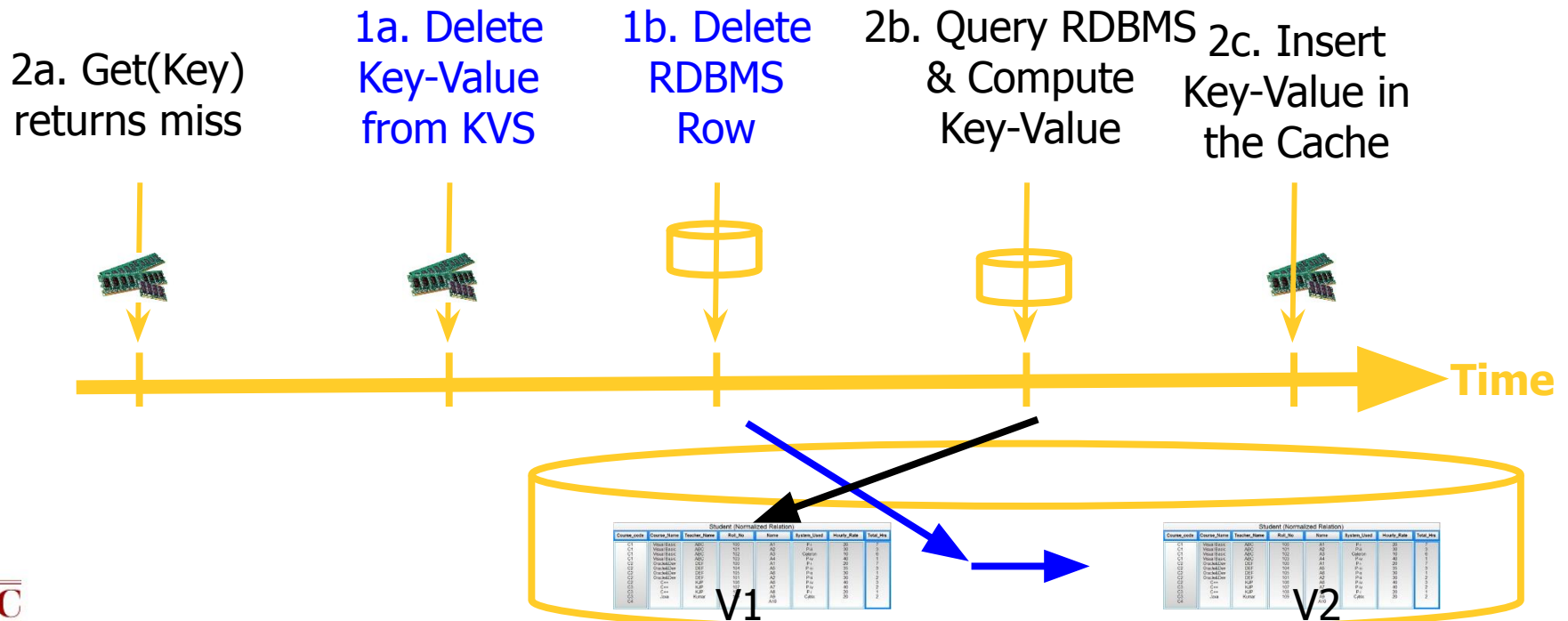
- May results in cache states that are *inconsistent* with the database state.



- Alice exists in the cache with no record in the RDBMS. Subsequent look ups for Alice observe cache hits, producing *stale* data.

Stale Data: Snapshot Isolation

- MVCC* may insert stale data.



Observations



- There is no network partition:
 - CAP Theorem does NOT apply.
- Preventing these limitations simplifies software development by making the behavior of programs more predictable:
 - One implementation for all application use cases.

Seeking a Solution?

CPU has caches. How does hardware maintain its CPU cache consistent with DRAM?

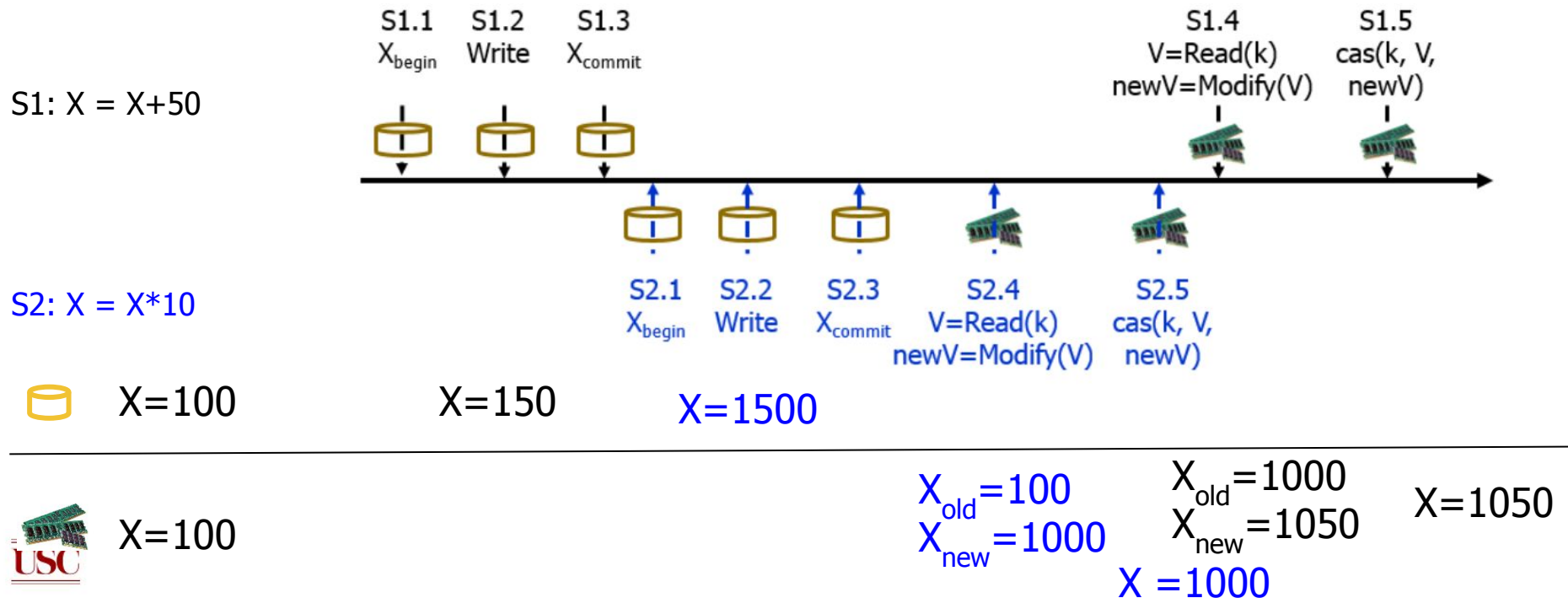
- Compare-and-Swap

Compare-and-Swap

- Compare-and-Swap, CaS, does not help. What is CaS?
 - Used by CPU caches
 - Prior to writing the value of a key, verify that its value has not changed since it was read.
- Why does not it help?
 - Consider two concurrent sessions:
 - Session 1 adds 50 to X, $X=X+50$
 - Session 2 multiplies X by 10, $X=X*10$
 - Assume the value of X is 100. Two possible serial schedules:
 - S1 S2, final value of X = 1500
 - S2 S1, final value of X = 1050

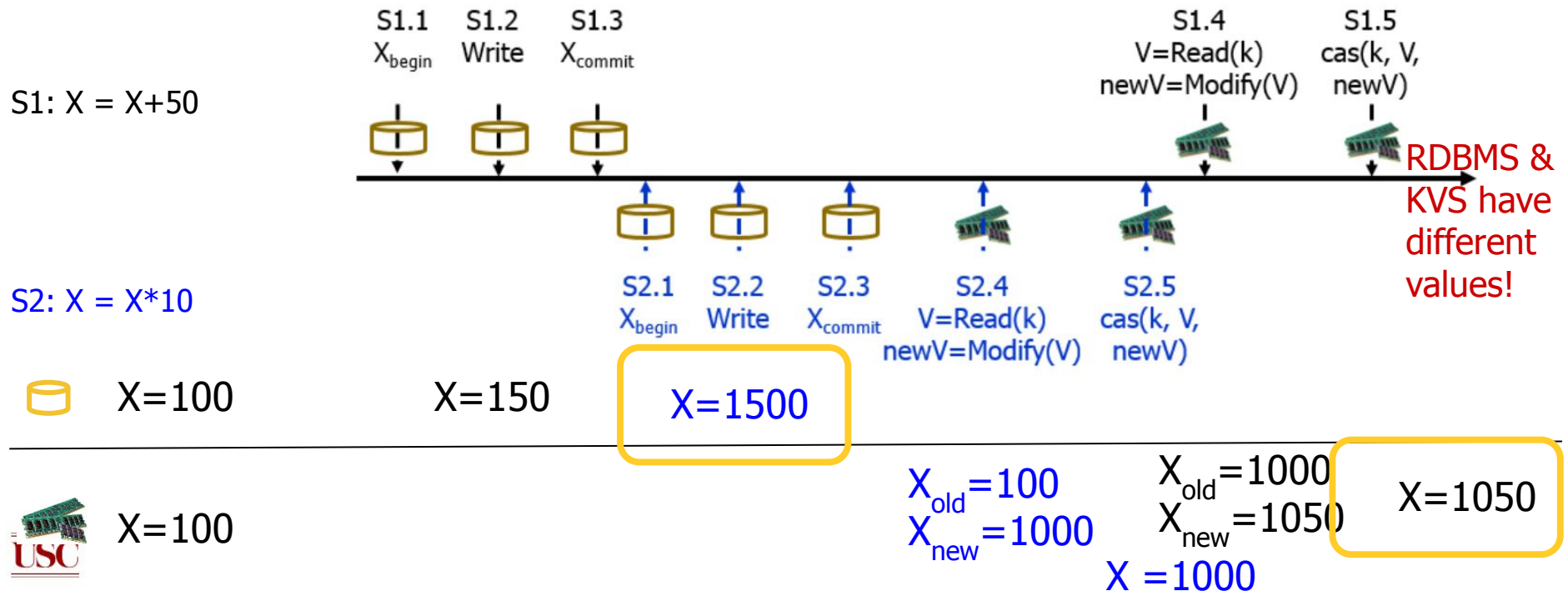
Compare-and-Swap

- Compare-and-Swap, CaS, does not help. What is CaS?
 - Prior to writing the value of a key, verify that its value has not changed since it was read



Compare-and-Swap

- Compare-and-Swap, CaS, does not help. What is CaS?
 - Prior to writing the value of a key, verify that its value has not changed since it was read



Questions?

Possible Approaches

1. Modify the Key-Value Store (KVS) only
2. Modify the RDBMS only
3. Modify both the RDBMS and the KVS [5]
4. One of the first 3 in combination with the application

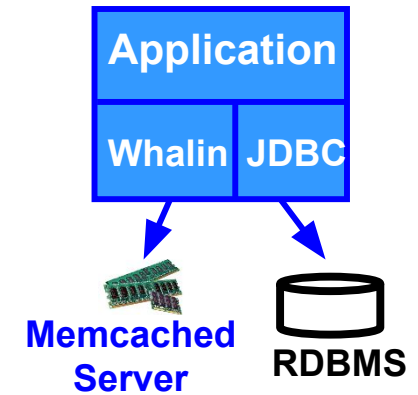
Possible Approaches

1. Modify the Key-Value Store (KVS) only
2. Modify the RDBMS only
3. Modify the RDBMS and the KVS [5]
4. One of the first 3 in combination with the application

Focus of the IQ framework: Approach 1.

Modify the KVS Only

- Changes to:
 - KVS client, e.g., Whalin client
 - KVS server, e.g., memcached
- Advantages:
 - Functions with diverse RDBMSs including proprietary ones, e.g., Oracle, Microsoft SQL Server, etc.
 - KVS components are simpler software systems than RDBMSs.
 - Enables transparent caching, e.g., KOSAR.



Terminology

- An Operation: Read (R), Write (W), Delete, Read-Modify-Write (R-M-W), δ using either the KVS or the RDBMS.

Operation	memcached command	SQL command
Read	get	SELECT ... FROM ... WHERE ...
Write	set	INSERT INTO tblname UPDATE tblname SET ... WHERE ...
Delete	delete	DELETE FROM tblname WHERE ...
R-M-W	get set/cas	
δ	append prepend	UPDATE tblname SET ... WHERE ...

Terminology: A Session



- A **session** is a sequence of operations consisting of at most one RDBMS transaction and one or more KVS operations. If the RDBMS aborts a session's transaction then the session aborts. KVS operations ordered after the transaction are ignored.
- A read session reads the value of one key. If it observes a miss then it queries the database to construct the value of the key and inserts the (key, value) in the cache.
- A write session performs an RDBMS write operation that may impact one or more key-value pairs, e.g., with invalidate, the session deletes the impacted key-value pairs from the KVS.
- KVS client and server implement the concept of a session. Session = Transaction with ACID semantics. Logically, a session imposes the serial schedule observed by the RDBMS onto the KVS.
- A session should observe its own updates. However, a session's updates should not be visible to other sessions until it commits.

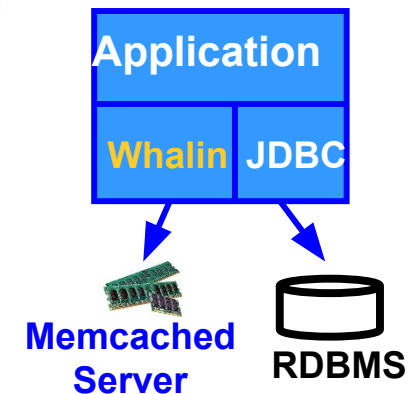
Questions?

IQ Framework with Invalidate

- Two leases:
 - Inhibit [9, 10]
 - Quarantine [9]

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q

- Implemented by the KVS client and server.
- Example: Whalin client and memcached.

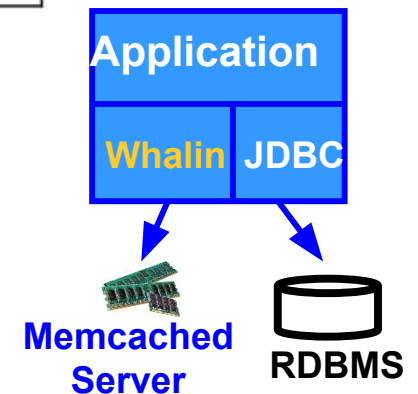


IQ Framework with Invalidate

- Two leases:
 - Inhibit [9, 10]
 - Quarantine [9]

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q

- I named Inhibit (I) and Quarantine (Q). The KVS grants an I lease to a session that observes a KVS miss, enabling it to query the RDBMS and populate the KVS with a key-value pair. The KVS grants at most one I lease on a key. Hence, at most one session may query the RDBMS to populate the KVS. All other concurrent read sessions referencing the same key must back off and try again. They observe either the value produced by the current lease holder or one of them will be granted an I lease to query the RDBMS and populate the KVS.
 - E
- d server.
ached.



©QL Systems. Middleware 2014.

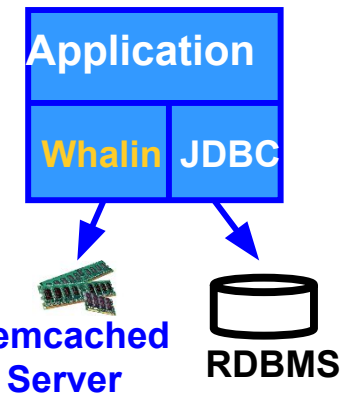
IQ Framework with Invalidate

- Two leases:
 - Inhibit [9, 10]
 - Quarantine [9]

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q

- A session that intends to either write, delete, δ , or R-M-W one or more keys must obtain a Q lease on each key explicitly. When a Q lease request for a key encounters an existing I lease held by a read session, S_R , the IQ framework invalidates the I lease of S_R to grant the Q lease. Subsequently, when S_R writes its computed value to the KVS, the KVS ignores its write operation because its I lease is no longer valid.

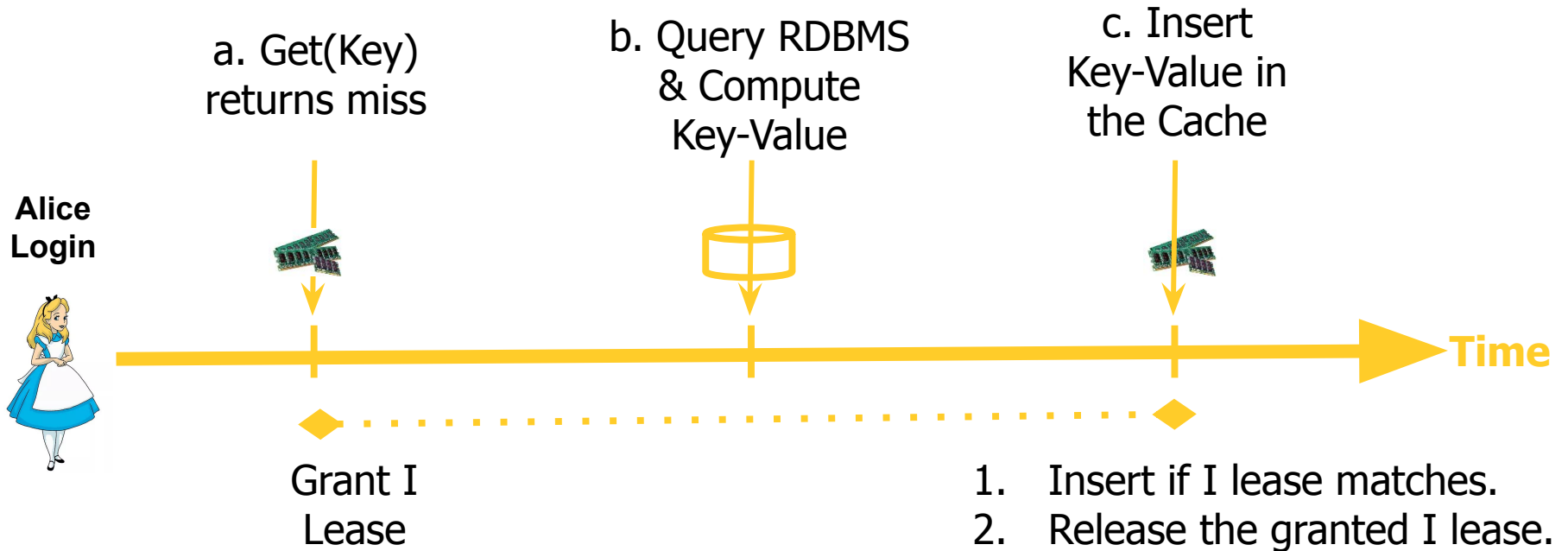
reverted.



is. Middleware 2014.

Inhibit Lease

- Granted to a cache miss.

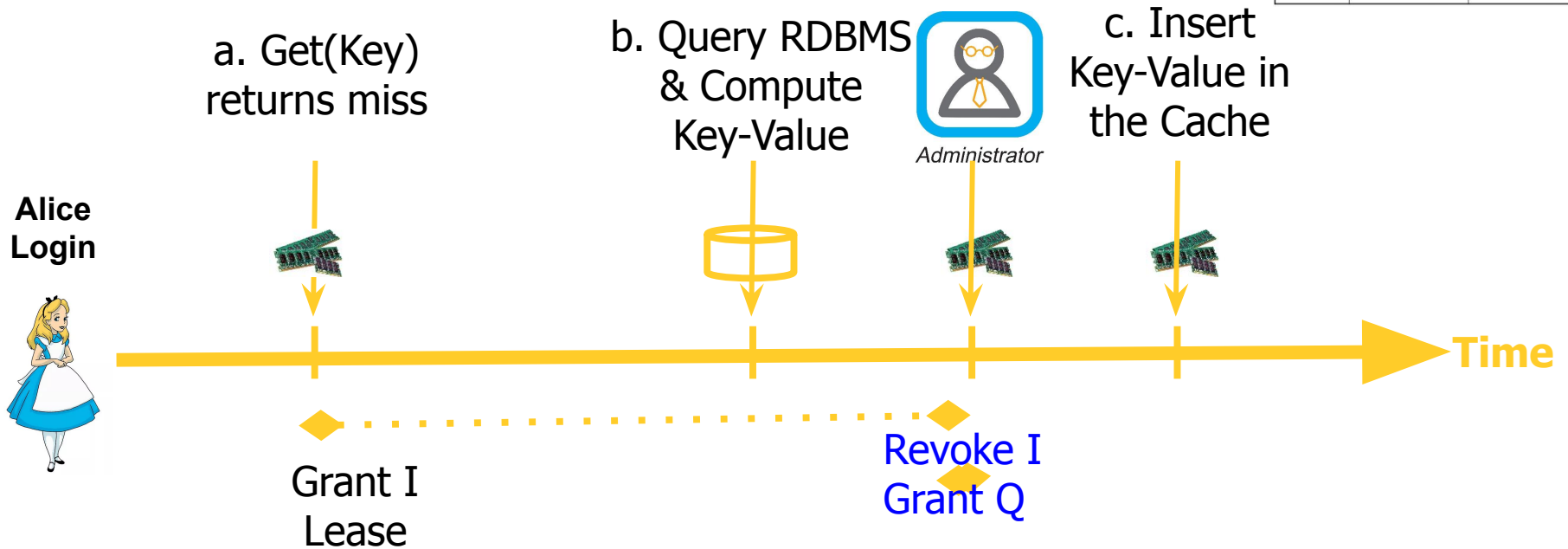


Inhibit & Quarantine Leases

- Granted to a cache miss.

1a. Delete
Key-Value
from KVS

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q

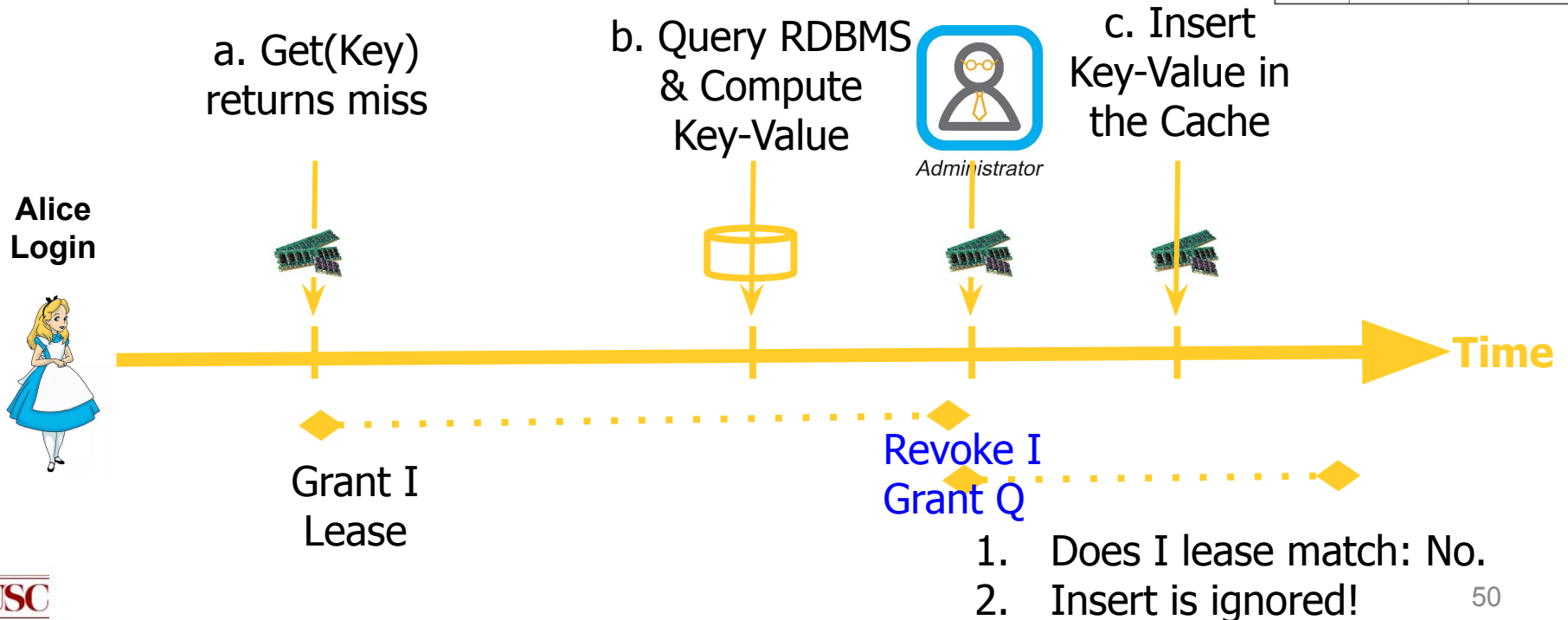


Quarantine Lease

- KVS ignores the inserts if I lease is not valid.

1a. Delete
Key-Value
from KVS

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q



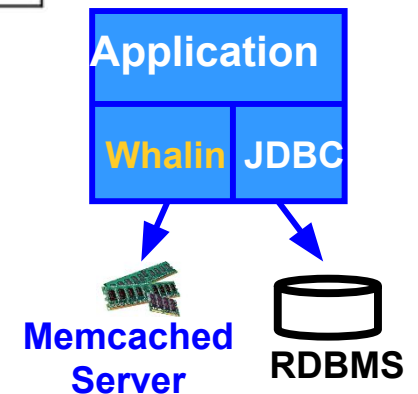
IQ Framework with Invalidate

- Two leases:
- Inhibit [9, 10]
- Quarantine [9]

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not possible, Back
Q	Grant I and void I	Grant Q

What is the difference between a lock and a lease?

- Implemented by the KVS client and server.
- Example: Whalin client and memcached.



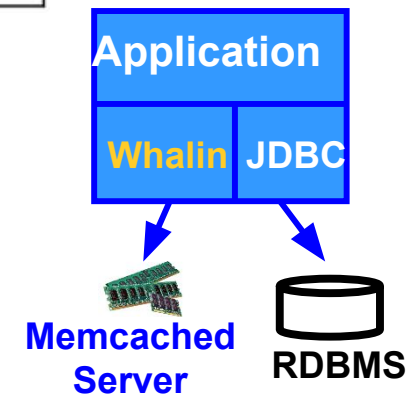
IQ Framework with Invalidate

- Two leases:
- Inhibit [9, 10]
- Quarantine [9]

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q

A lease for a key has a fixed life time and is granted to one KVS connection (thread) at a time. The finite life time enables the KVS to release the lease and continue processing operations in the presence of node failures hosting the application. This is particularly true with refresh and δ due to how they use the Q leases: If the KVS holds Q leases indefinitely (similar to locks) then both node failures and intermittent network connectivity may degrade system performance severely. With a finite life time for leases, the KVS recovers from node failures hosting an application with granted leases.

recovered.



ns. Middleware 2014.

IQ Framework with Invalidate

- Two leases:
- Inhibit [9, 10]
- Quarantine [9]

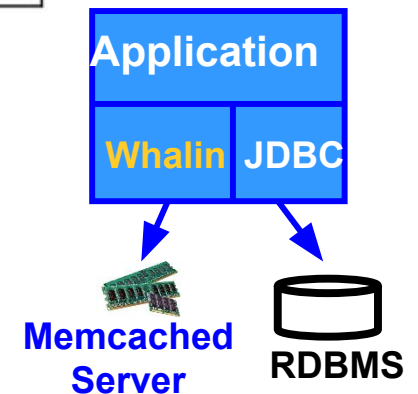
Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Grant Q, void I
Q	Grant Q, void I	Grant Q

What happens when the life time of a lease is too short?

- In
- Exam

A lease for a finite life time and is granted to one thread (thread) at a time. The finite life time operations in the presence of node failures hosting the application. This is particularly true with refresh and δ due to how they use the Q leases: If the KVS holds Q leases indefinitely (similar to locks) then both node failures and intermittent network connectivity may degrade system performance severely. With a finite life time for leases, the KVS recovers from node failures hosting an application with granted leases.

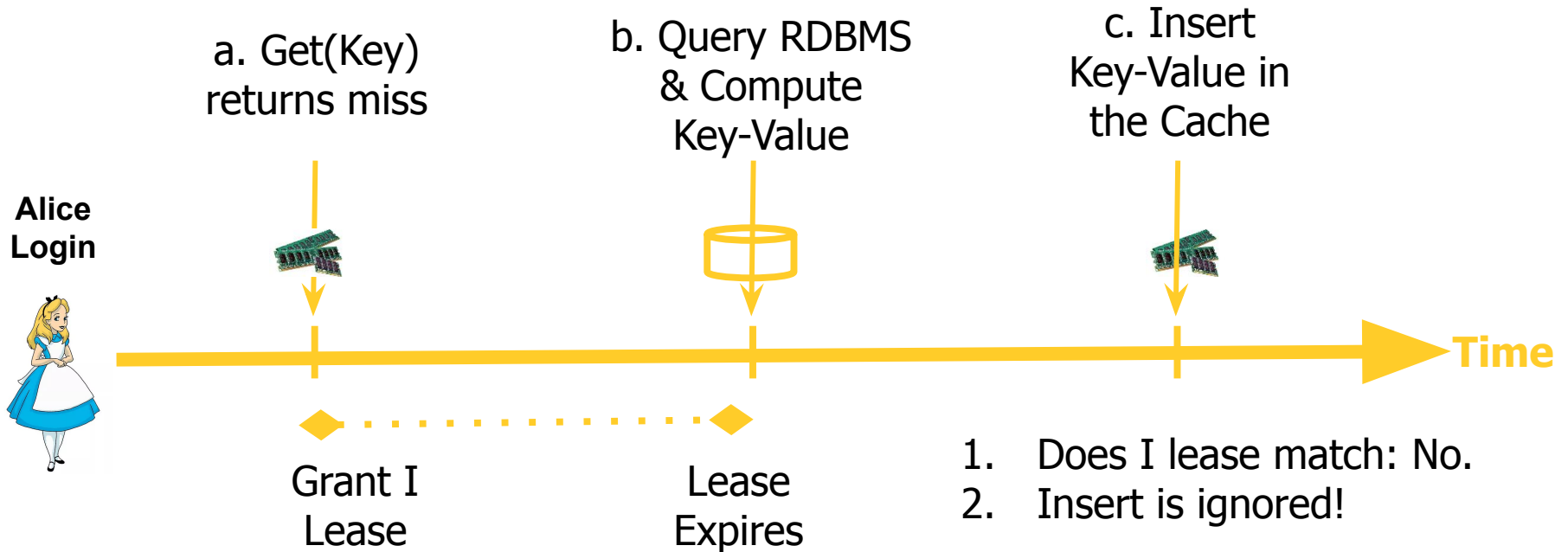
server.
ed.



ns. Middleware 2014.

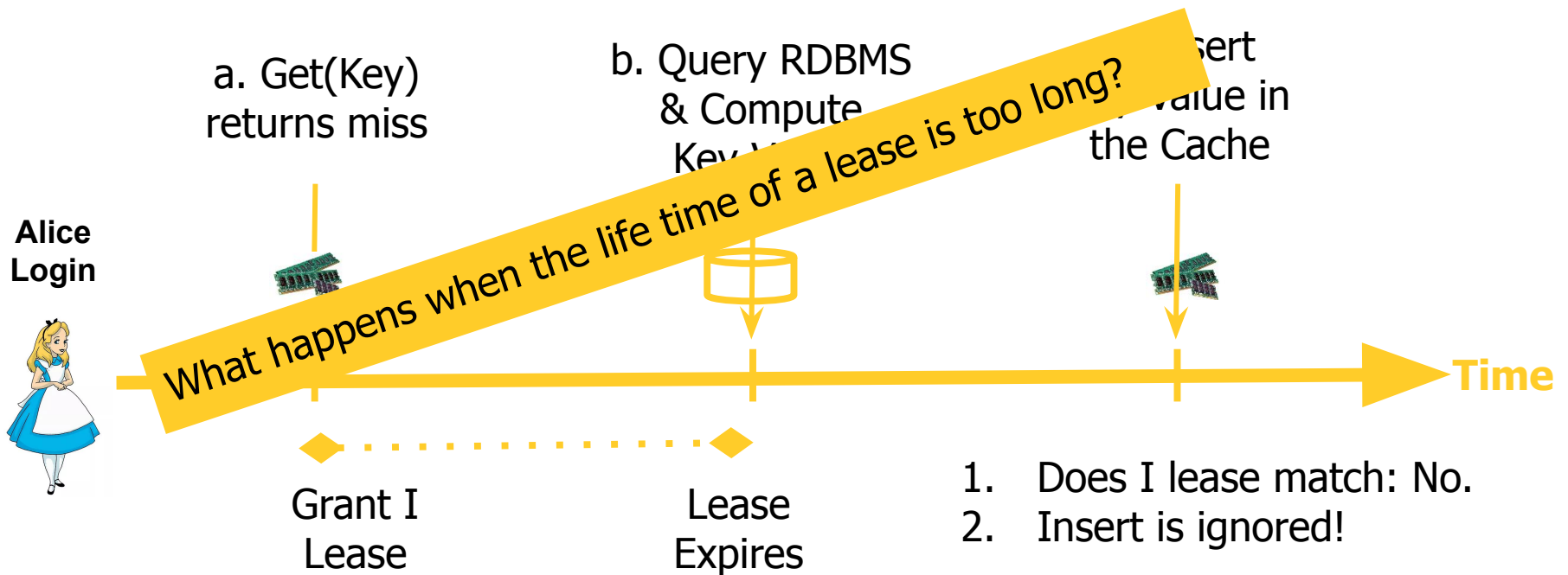
Lease with a Short Life Time

- An empty cache!



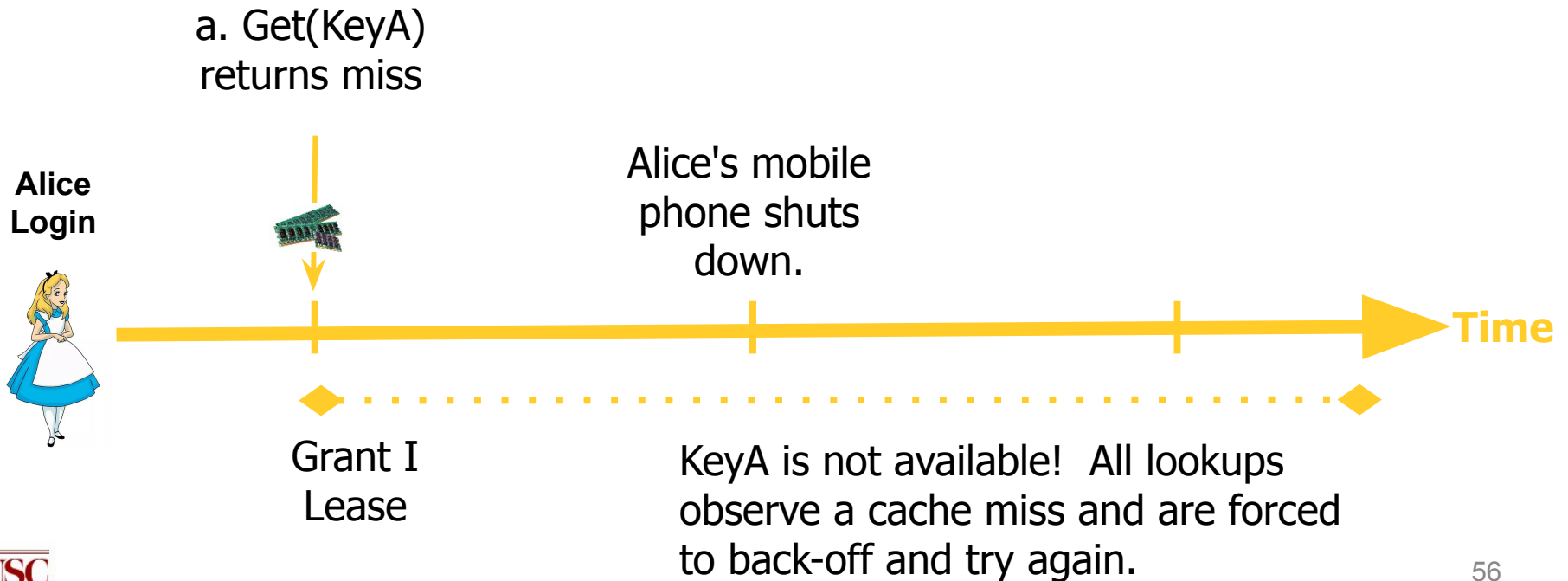
Lease with a Short Life Time

- An empty cache!



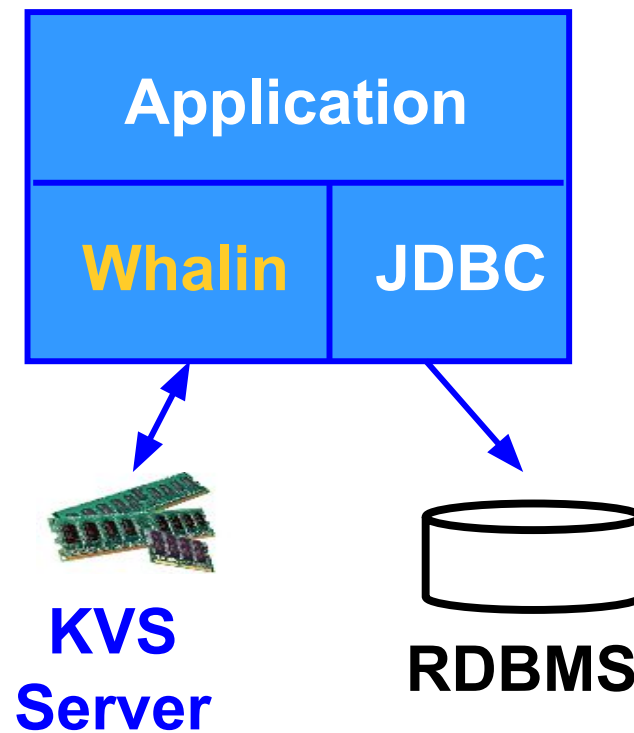
Lease with a Long Life Time

- In the presence of failures, a key-value pair may become unavailable for the long life of the lease



How is the I-Lease implemented?

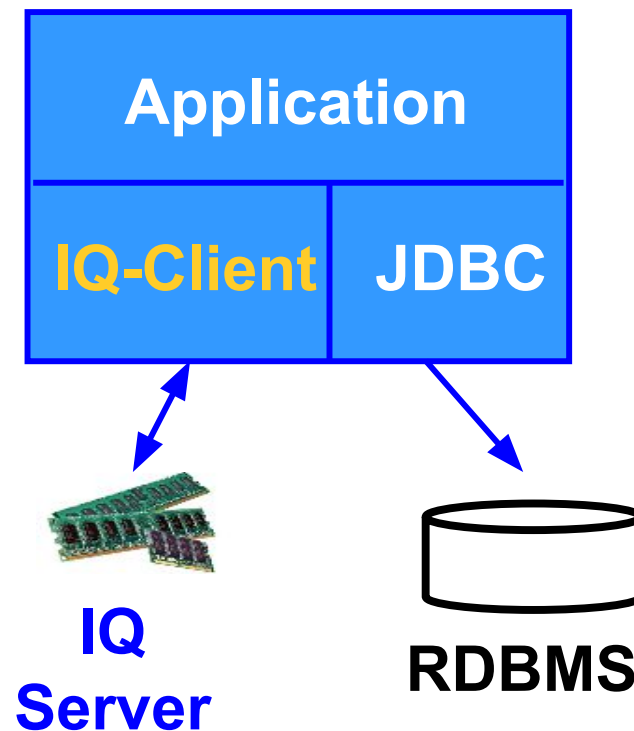
A lot of design details may be implemented in the KVS client, Whalin, making them transparent to the application.



How is the I-Lease implemented?

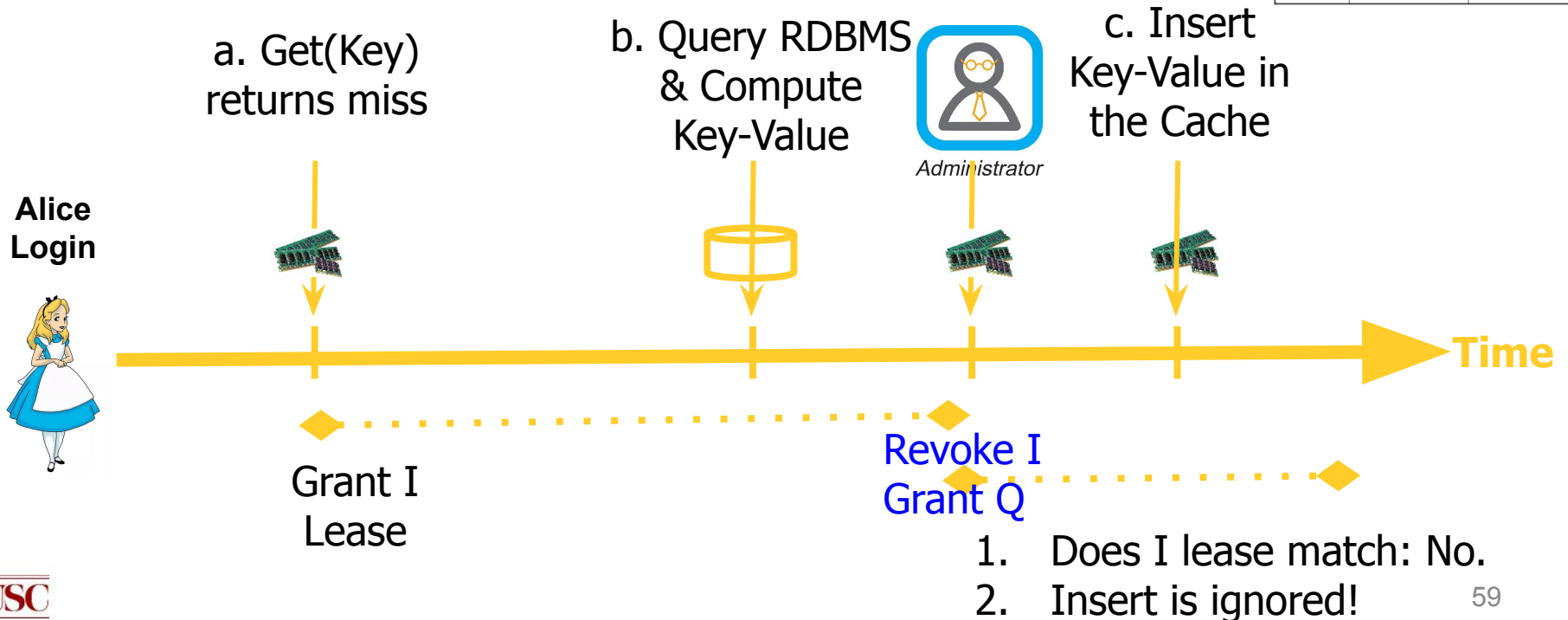
IQ-Server represents a lease as a token generated by the KVS server.

IQ-Client maintains the lease granted by the KVS server. When a set is issued by the application, IQ-Client provides the I lease to the IQ-server as an argument of the set command.



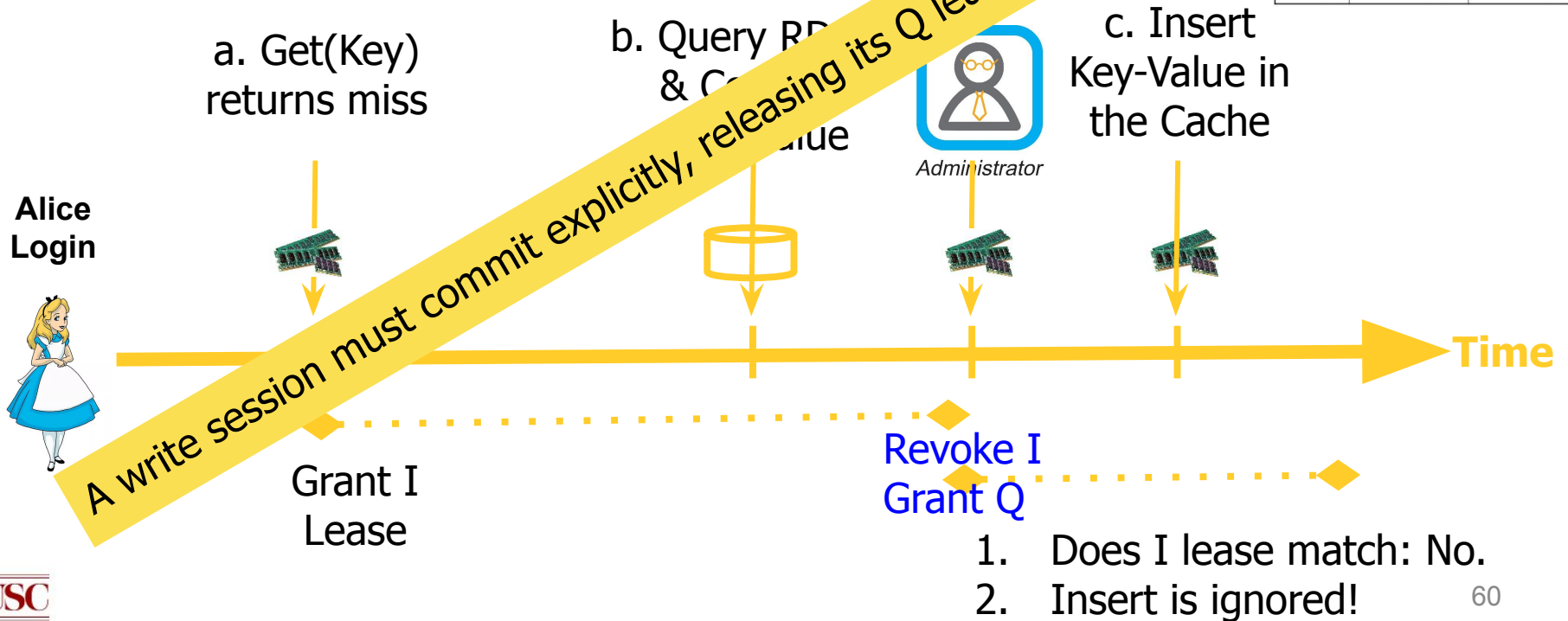
When is Quarantine lease released?

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q



When is Quarantine lease released?

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q



USC Viterbi
School of Engineering

1. Does I lease match: No.
2. Insert is ignored!

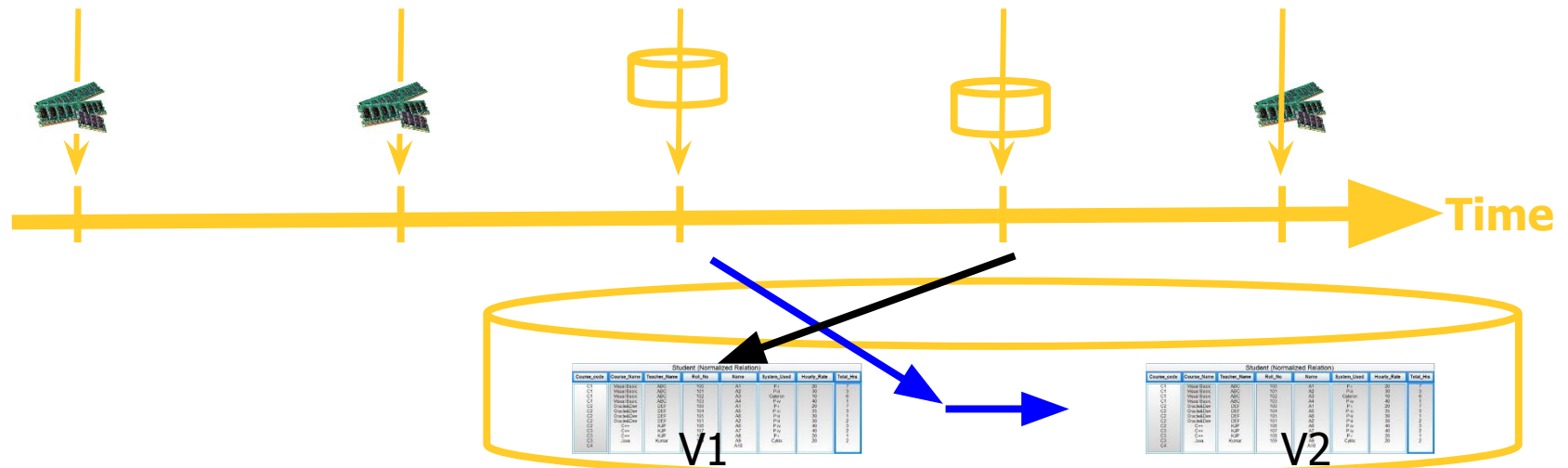
Revoke I
Grant Q

1a. Delete Key-Value from KVS

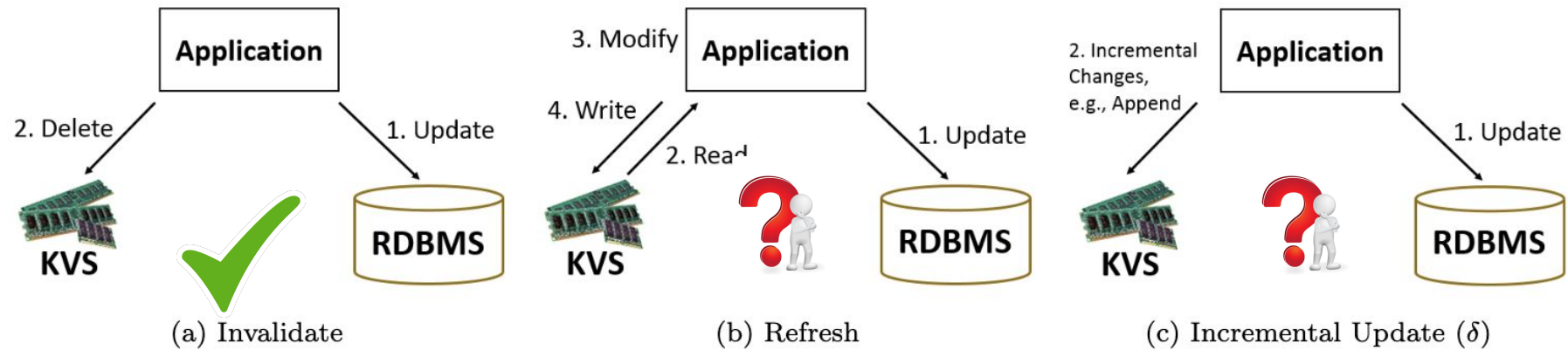
1b. Delete RDBMS Row

2b. Query RDB & Compute Key-Value

IS 2c. Insert Key-Value in the Cache



What about Refresh, Incremental Update?

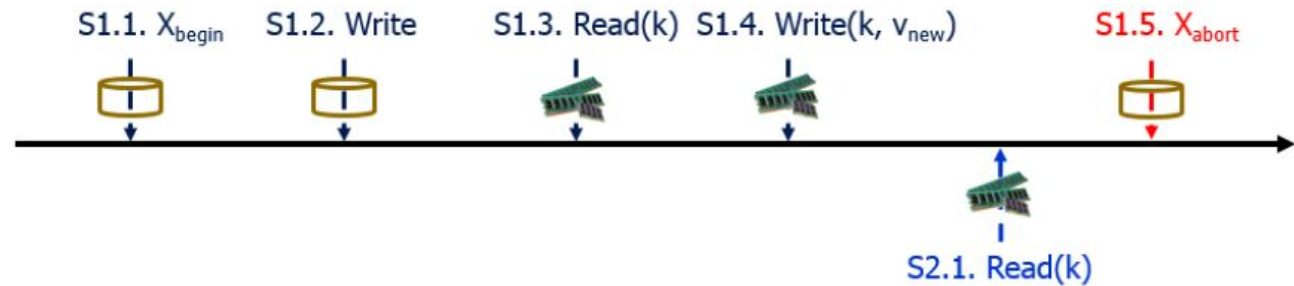


System Load	Invalidate	Refresh	Incremental Update
1 session	0%	0%	0%
Low 10 concurrent sessions	0.5%	0%	0.01%
Moderate 100 concurrent sessions	1.1%	1.4%	0.2%
High 200 concurrent sessions	1.3%	1.8%	2.9%

Table 1: Percentage of unpredictable data with Twemcache. These percentages are reduced to zero with the IQ framework.

What is the BIG deal?

Dirty read with
refresh:



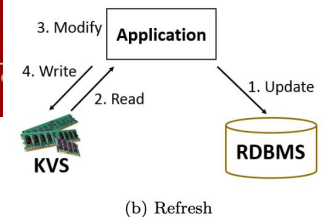
Step S1.3. obtains a Q lease.

Step S2.1. observes a cache hit and does not require an I lease.

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q

- A session is required to:
 - a. Obtain Q leases on those key value pairs that it intends to update prior to committing its RDBMS transaction.
 - b. Write its updated key-value pairs to the KVS after its RDBMS transaction commits and release its Q leases at session commit time.
- Once the KVS grants a Q lease on a key-value pair and the lease expires, the KVS deletes the key-value pair.
 - a. To handle different form of failures. For example, consider what happens when a session loses its network connection in middle of writing its updated key-value pairs to the KVS.

Refill: Requirements



- Quarantine-and-Read, QaRead(key)
 - Acquires a Q lease on the referenced key from the server and reads the value of the key (if any).
 - If another session has a Q lease on the key then the requesting session aborts and retries.
- Swap-and-Release, SaR(key, vnew)
 - Issued after RDBMS transaction commit.
 - Changes the current value of the specified key with the new value, vnew, and releases its Q lease.

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Reject and Abort requester

- The requirements resemble 2-phase commit:
 - a. A session issues all its QaRead calls prior to the RDBMS transaction commit, i.e., obtains Q leases.
 - b. A session issues all its SaR invocations after RDBMS transaction commit, i.e., updates KVS and releases Q leases.

Invite Friend (InviterID, InviteeID)

1. Key = "Profile"+InviteeID
2. $V_{old} = \text{QaRead (Key)}$
3. $V_{new} = \text{Increment } V_{old}.\#PendingFriends$
4. Begin RDBMS Xact
 - a. Insert (InviterID, InviteeID, 1) in PendingFriends
 - b. Update PendingCount of invitee by 1 in Users table
5. Commit Xact
6. SaR (Key, V_{new})

Questions?

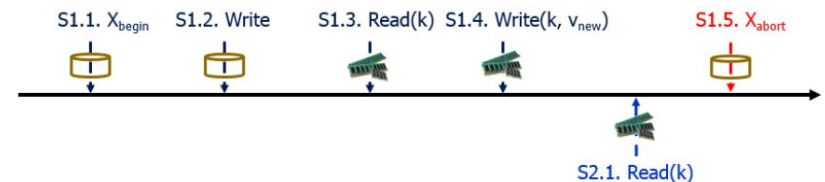
Question

- The requirements resemble 2-phase commit:
 - a. A session issues all its QaRead calls prior to the RDBMS transaction commit, i.e., obtains Q leases.
 - b. A session issues all its SaR invocations after RDBMS transaction commit, i.e., updates KVS and releases Q leases.

How does QaRead and SaR prevent this dirty read?

Invite Friend (InviterID, InviteeID)

1. Key = "Profile"+InviteeID
2. $V_{old} = \text{QaRead (Key)}$
3. $V_{new} = \text{Increment } V_{old}.\text{\#PendingFriends}$
4. Begin RDBMS Xact
 - a. Insert (InviterID, InviteeID, 1) in PendingFriends
 - b. Update PendingCount of invitee by 1 in Users table
5. Commit Xact
6. SaR (Key, V_{new})



Refresh/Refill

Invite Friend (InviterID, InviteeID)

1. Key = "Profile"+InviteeID
2. $V_{old} = \text{QaRead (Key)}$
3. $V_{new} = \text{Increment } V_{old}.\#PendingFriends$
4. Begin RDBMS Xact
 - a. Insert (InviterID, InviteeID, 1) in PendingFriends
 - b. Update PendingCount of invitee by 1 in Users table
5. Commit Xact
6. SaR (Key, V_{new})

Invite Friend (InviterID, InviteeID)

1. Begin RDBMS Xact
 - a. Insert (InviterID, InviteeID, 1) into Friendships table
 - b. Update PendingCount of invitee by 1 in Users table
 - c. Key = "Profile"+InviteeID
 - d. $V_{old} = \text{QaRead (Key)}$
 - e. $V_{new} = \text{Increment } V_{old}.\#PendingFriends$
2. Commit Xact
3. SaR (Key, V_{new})

(a) KVS operations prior to the RDBMS transaction (b) KVS operations during the RDBMS transaction

Workload	QaRead calls prior to RDBMS transaction start		QaRead calls during RDBMS transaction	
	Avg	Max	Avg	Max
0.1%	2	4	0	0
1%	6.02	74	1.18	5
10%	4.61	77	1.33	9

Paradigm (b) Reduces
duration of time a
session holds onto a Q
lease.

Refresh/Refill: Challenge

Invite Friend(InviterID, InviteeID)

1. Begin RDBMS Xact

- a. Insert(InviterID, InviteeID, 1) in PendingFriends
- b. Update PendingFriendCount of InviteeID by 1 in the Users table
- c. Key1 = "Profile"+InviteeID
- d. Vold1 = QaRead(key1)
- e. Vnew1 = Increment Vold.#PendingFriends
-
- k. Vold2 = QaRead(Key1)

Refresh does
not observe its
own write!

Not acceptable!

2. Commit Xact

3. SaR(Key, Vnew)

What is the solution?

Refresh/Refill: Challenge

Invite Friend(InviterID, InviteeID)

1. Begin RDBMS Xact

- a. Insert(InviterID, InviteeID, 1) in PendingFriends
- b. Update PendingFriendCount of InviteeID by 1 in the Users table
- c. Key1 = "Profile"+InviteeID
- d. Vold1 = QaRead(key1)
- e. Vnew1 = Increment Vold.#PendingFriends
-
- k. Vold2 = QaRead(Key1)

Refresh does
not observe its
own write!

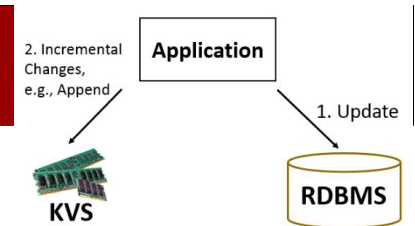
Not acceptable!

2. Commit Xact

3. SaR(Key, Vnew)

Do not allow write sessions to do cache lookup. They must issue RDBMS queries. Must exercise care because a transaction's update to the RDBMS is visible to itself.

Incremental Update: Requirements



- Quarantine-and-update, $IQ-\Delta(k_i, \delta_i)$
 - Δ is an incremental update such as append or increment
 - δ_i is the change to the key, e.g., value being appended.
- The KVS buffers δ_i , maintaining the old value of k_i .
- Once the session commits, the KVS applies δ_i to k_i 's value.
- If the session aborts, the KVS discards δ_i .

Evaluation using BG

IQ vs. I

System Load	Mix of Actions	10K members		100K members	
		Invalidate	Refresh	Invalidate	Refresh
Low, 10 Threads	0.1%	0.6%	0%	0%	3.3%
	1%	0.5%	0%	0%	3.5%
	10%	0.2%	0%	0%	3.1%
Moderate, 100 Threads	0.1%	1.7%	0.02%	0%	3.3%
	1%	1.1%	1.4%	0%	3.4%
	10%	0.9%	6.3%	0%	3%
High, 200 Threads	0.1%	2.0%	0%	0%	3.2%
	1%	1.3%	1.8%	0%	3.4%
	10%	1.3%	8.3%	0%	2.8%

Table 7: Percentage of unpredictable data using invalidate/refresh with Twemcache by itself. These percentages are reduced to zero with the IQ-Twemcached

Evaluation: IQ vs. I

Overhead of Q leases is minimal (if any):

	Invalidate		Refresh	
	Twemcache	IQ Twemcached	Twemcache	IQ Twemcached
0.1%	31,492	31,473	31,338	31,184
1%	31,144	31,246	30,615	30,352
10%	29,317	29,204	29,194	29,277

Table 8: SoAR of Twemcache and IQ-Twemcached with a 100% utilized CPU.

Conclusion

IQ Framework:

- Non-blocking and deadlock free.
- Availability of data in the presence of failures.
- Compatibility matrix between I and Q with Invalidate is different than with Refresh/Incremental update.

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Grant Q

(a) Invalidate

Existing Lease \ Requesting Lease	I	Q
I	KVS miss, Back off	Not compatible, Back off
Q	Grant Q and void I	Reject and Abort requester

(b) Refresh

Questions?

Thundering Herds

- A thundering herd happens when a specific key undergoes heavy read and write activity.
 - Writes invalidate the cache repeatedly
 - All reads are forced to query the database
- I lease solves this problem
 - The first read for the specific key is granted the I lease
 - All other reads observe a miss and back-off
 - The read with the I lease queries the RDBMS, computes the missing value, and populates the cache
 - All other reads observe a cache hit

